

Tutor: Good afternoon! Let's start with the warm-up: in your own words, what's the difference between `x = 5` and `x == 5`—and why does that distinction matter?

Student: one equal sign denotes an assignment, in this case 5 is assigned to variable x. two equal signs denotes a boolean truth, that x is 5.

Tutor: Exactly—`=` is a command that assigns a value, while `==` asks a question and produces a Boolean result. Why does it matter that Python treats those as two fundamentally different operations?

Student: because they do not say the same thing

Tutor: Right—that's the key distinction: one changes/sets something, while the other checks something. If `x` were currently 7, what would you expect `x = 5` to do versus `x == 5`?

Student: `x=5` would change the variable assignment, and `x==5` would be false

Tutor: Yep, exactly. Now Problem 1: given `a = 5`, `b = "5"`, `c = 5.0`, and `d = True`, what do you predict `type()` returns for all four before checking?

Student: for a it would put 5, for b it would put "5", for c it would put 5, for d it would put `a==5`

Tutor: You've predicted the values, which is a useful distinction to catch. `type()` asks what kind of value each one is, so what types do you predict for `a`, `b`, `c`, and `d`?

Student: oh type outputs text not integers. so for a it would give an error, b would be 5, im not sure about c, and for d it would be 5?

Tutor: Close, but `type()` doesn't output the value—it identifies its type. Think of the four primitive types from class: integer, string, float, and Boolean; which one would you assign to each of `a`, `b`, `c`, and `d`?

Student: oh okay. so a is integer, b is a string, c is float, and d is boolean

Tutor: Yep, all four are right. Now predict before checking: what Boolean results do you expect from `a == c`, `a == b`, and `d == 1`?

Student: `a==c` is true, `a==b` is false, and `d==1` is true

Tutor: Exactly—all three predictions are right. Notice especially that `a == b` quietly gives `False` rather than an error; where else have you seen a wrong answer arrive with no error?

Student: when you used `type()`

Tutor: That's a reasonable connection, but I'm thinking back to Session 1, when code ran cleanly yet gave the wrong result. Do you remember what small issue in the data caused that?

Student: there was an extra space after the variable for some of the samples. so it missed the samples that were variable a space

Tutor: Exactly—the trailing space made those values different even though they looked like they should match, so the code could run and still be wrong. Now Problem 2: suppose `age_str = "42"` and then we try `age_str + 8`; predict first—which possibility do you expect: a `TypeError`, `"428"`, or `50`?

Student: probably a type error since 8 is an integer and 42 is treated as a word

Tutor: Yep—that prediction is right: Python refuses to guess how a string and an integer should be combined. Is that refusal a good thing or a bad thing here, and why?

Student: good thing because it could give you the wrong answer you are looking for

Tutor: Exactly—the error is useful because a silent result could look valid while being scientifically wrong. What do you think whoever wrote this intended the result to be, and what would need to change conceptually to make that happen?

Student: i think they meant to add 42 and 8. so they would need to write `int(age_str) + 8`

Tutor: Yes—that matches the likely intent: treat `"42"` as the number 42 before adding. Before we move on, what result do you predict that expression would produce?

Student: 50

Tutor: Right, 50. Now Problem 3—the important part is reading the traceback from the bottom first:

Traceback (most recent call last):

File "clean.py", line 27, in

`total = ages.sum() + max_age`

`TypeError: unsupported operand type(s) for +: 'int' and 'str'`

What is that bottom line actually complaining about?

Student: it cannot add the `ages.sum() + max_age` because one is an integer and one denotes a string

Tutor: Exactly—the `+` is being asked to combine an integer and a string, and Python refuses. Which of the two things being added do you think is the string, and how could you find out?

Student: you could find out by just typing one at a time to see what it gives you?

Tutor: Yes—that could help, especially if you inspect what each variable contains and what type it has. Before checking, which do you predict is the string: `ages.sum()` or `max_age`?

Student: i think `max_age` is a string

Tutor: Yep, that's the likely culprit. Line 27 is where Python crashed—but does that necessarily mean line 27 is where the actual bug happened?

Student: no

Tutor: Exactly—the bad type could have been introduced earlier, and line 27 is just where it finally caused a crash. If `max_age` was supposed to be numeric, where upstream would you start looking for the bug?

Student: at the beginning

Tutor: Yes—especially where `max_age` first gets its value or enters the program. If the age data contained several entries like `"unknown"`, at what point would you want to notice that the ages weren't all numeric?

Student: at the beginning

Tutor: Exactly—when the data first comes in, before later calculations depend on it. Let's jump to Problem 6: suppose an AI makes the `TypeError` disappear by converting `max_age` to a string, but your “total” becomes `"4210"` instead of `52`; did the AI actually fix the bug?

Student: no

Tutor: Right—the error disappeared, but the result still violates what you intended. What did the AI know from the error message, and what couldn't it know about what `max_age` was supposed to represent?

Student: it knew that they were different types and needed to be the same type in order to run. it didnt know that you wanted to add them numerically

Tutor: Exactly—that's the key lesson: the AI could see the type mismatch, but it couldn't infer your scientific intent just from the error. What could you tell the AI in your prompt so it investigates why `max_age` became a string rather than merely silencing the error?

Student: im not sure

Tutor: Think about stating your expectation explicitly rather than asking only how to remove the error. How would you finish this prompt: “I expect `max_age` to be _____. Help me figure out where it became _____.”

Student: an integer; a string

Tutor: Exactly: “I expect `max_age` to be an integer; help me figure out where it became a string.” That gives the AI information about your intent, not just the error.

For our wrap-up, in one sentence: what’s the one thing about types you want to remember when you paste code from an AI?

Student: i want to remember that AI doesn't know what i want, it only knows how to make the code run

Tutor: Exactly—that’s the takeaway: code that runs isn’t necessarily code that is correct, and you have to supply the intent.

Before you leave, paste this whole conversation into the Session 2 participation assignment on Brightspace as-is, including anywhere we went in circles. Credit is for engaging with the questions, not for getting everything right.